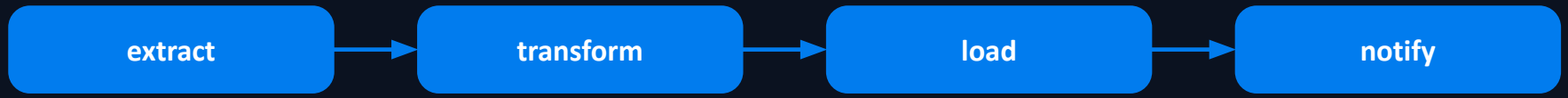
Apache Airflow in 90 minutes

From zero to a working ETL DAG, running on your laptop



Databurst · hands-on session · Airflow 2.9

Agenda: six sections, six DAGs

1	**Architecture & core concepts**	5–15 min	`01_hello_airflow`
2	**Writing DAGs: TaskFlow & XCom**	15–30 min	`02_taskflow_etl`
3	**Scheduling, intervals & templating**	30–45 min	`03_scheduling_and_templating`
4	**Control flow: mapping, branching, groups**	45–60 min	`04_control_flow`
5	**Connections, hooks & sensors**	60–75 min	`05_connections_and_sensors`
6	**Production patterns & wrap-up**	75–90 min	`06_production_patterns`

Why not just cron and a few scripts?

cron + scripts

- Step B runs at 02:30 and hopes step A finished at 02:00
- A failure at 3am is an email nobody reads
- Re-running last Tuesday means editing dates by hand
- Secrets live in the script, the box, or someone's shell history
- “Is it running? What did it do yesterday?” → ssh and grep
- Scaling = another box, another crontab

an orchestrator

- Explicit dependencies: B starts when A succeeds
- Retries, timeouts and alerts are configuration, not code
- Backfill: “re-run every day since 2026-10-01” is one click
- Connections & secrets are managed centrally
- UI + API: state, logs and history for every run
- Workers scale independently of the definitions

Airflow: workflows as Python code

2014

born at Airbnb, Apache top-level project since 2019

2.9

released April 2024; this course runs 2.9.3, the last 2.9 release

90+

provider packages: AWS, GCP, Azure, Snowflake, dbt, Slack, …

A DAG (directed acyclic graph) is a Python file that declares **tasks** and the **dependencies** between them. Airflow schedules runs, executes tasks in order, retries failures and records everything.

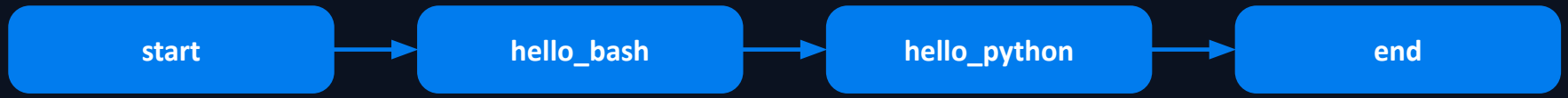


Good for: batch pipelines, ML training jobs, report generation, anything scheduled with dependencies.
Not for: streaming / sub-second latency, moving large data through the scheduler itself.

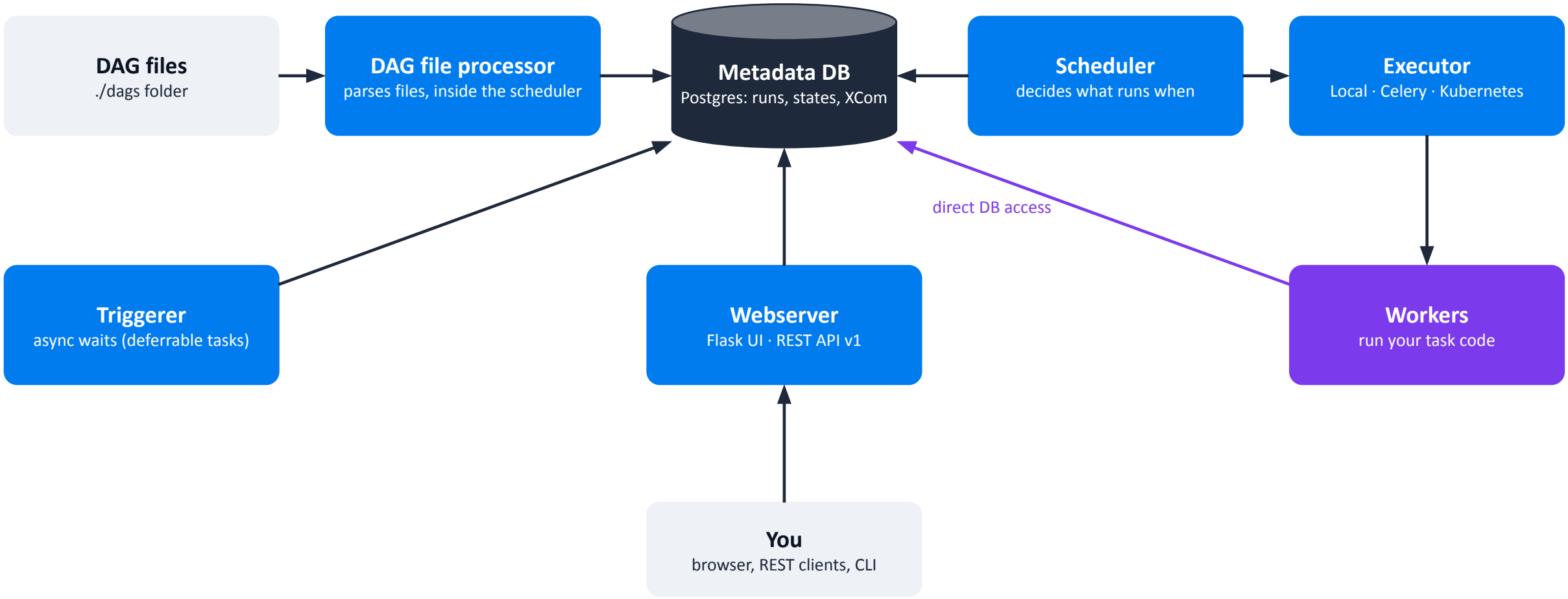
SECTION 1 · 5–15 min

Architecture & core concepts

Demo DAG `01_hello_airflow`



Architecture: what runs where



Airflow 2: workers, the triggerer and the webserver all read and write the metadata DB directly.

Six words you need

DAG

A Python file declaring tasks and their order. One dag_id, one schedule. Directed, acyclic: no loops.

Task

One unit of work inside a DAG: run a script, a query, a sensor. Identified by task_id.

Operator

A reusable task template: BashOperator, SQLExecuteQueryOperator, S3 and Slack operators. @task wraps your own Python.

DAG Run

One execution of the whole DAG for one logical date / data interval. Scheduled by its timetable or triggered manually.

Task Instance

One task inside one DAG run, with a state: queued → running → success / failed / skipped … and a try number.

XCom

“Cross-communication”: a small value a task returns and another task reads. Lives in the metadata DB.

Demo 1: hello_airflow and the UI tour

```
@dag(
    dag_id="01_hello_airflow",
    schedule=None,          # manual only
    start_date=pendulum.datetime(2026, 1, 1, tz="UTC"),
    catchup=False,
    tags=["databurst", "section-1"],
)
def hello_airflow() -> None:
    start = EmptyOperator(task_id="start")
    hello_bash = BashOperator(
        task_id="hello_bash",
        bash_command='echo "Hello from $(hostname)"',
    )

    @task
    def hello_python() -> str:
        ctx = get_current_context()
        print(f"run_id={ctx['run_id']}")
        return "hello"

    end = EmptyOperator(task_id="end")
    start >> [hello_bash, hello_python()] >> end

hello_airflow()
```

DAGs list

unpause the toggle, read tags and the next run

Grid view

runs as columns, tasks as rows, colour = state

Graph view

the dependencies you wrote, mapped tasks expand

Trigger

the ▶ button; a run appears in seconds

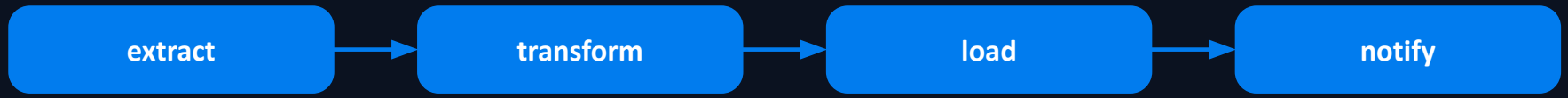
Logs

click a task instance → Logs tab; also Rendered template and XCom

SECTION 2 · 15–30 min

Writing DAGs: TaskFlow & XCom

Demo DAG `02_taskflow_etl`



Operators and @task: two kinds of task, one DAG style

Operators

```
@dag(schedule="@daily", start_date=..., catchup=False)
def etl() -> None:
    extract = SQLExecuteQueryOperator(
        task_id="extract",
        conn_id="warehouse",
        sql="SELECT count(*) FROM orders",
    )
    report = BashOperator(
        task_id="report",
        bash_command="echo {{ ti.xcom_pull('extract') }}",
    )
    extract >> report


etl()
```

TaskFlow @task

```
from airflow.decorators import dag, task

@dag(schedule="@daily", start_date=..., catchup=False)
def etl() -> None:

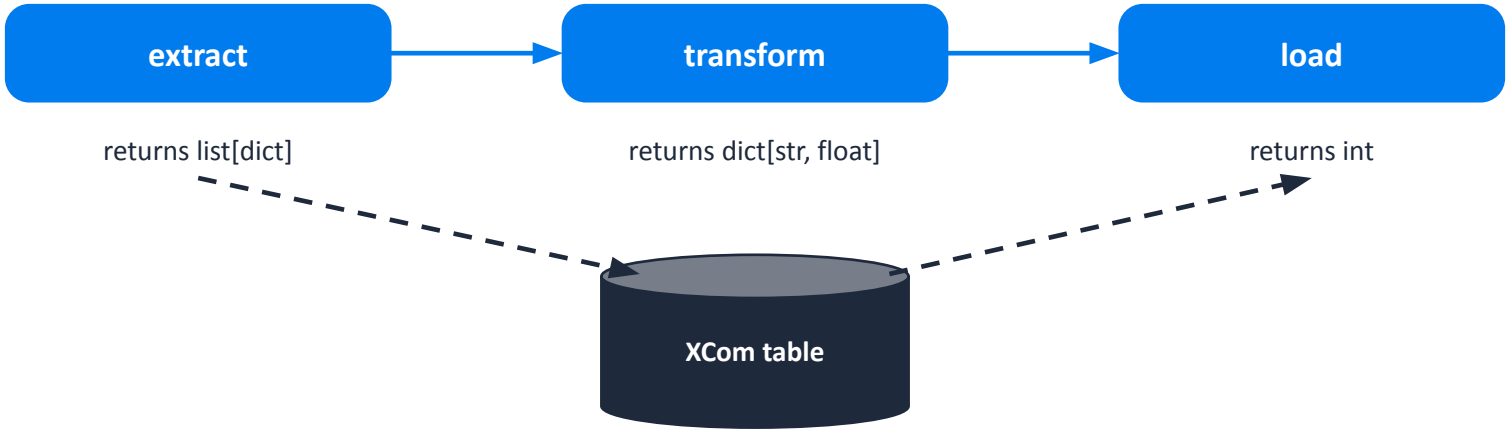
    @task
    def extract() -> list[int]:
        return [1, 2, 3]

    @task
    def load(rows: list[int]) -> None:
        print(len(rows))

    load(extract())      # dependency + XCom


etl()
```

XCom: how data moves between tasks




```
# in templates (classic operators)
bash_command='echo "{{ ti.xcom_pull(task_ids=\'load\') }}"'

# explicit push / pull inside any task
ti = get_current_context()["ti"]
ti.xcom_push(key="rows", value=42)
ti.xcom_pull(task_ids="extract", key="rows")

# several values from one task
@task(multiple_outputs=True)
def split() -> dict[str, int]: return {"a": 1, "b": 2}    # split()["a"]
```

Good for

Row counts, file paths, S3 keys, small dicts, a status flag. Anything JSON-serialisable and small (kilobytes).

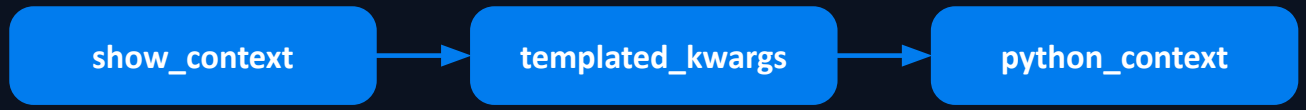
Not for

DataFrames, files, query results. Write those to storage and pass the path or key instead. XCom goes through the metadata DB on every push and pull.

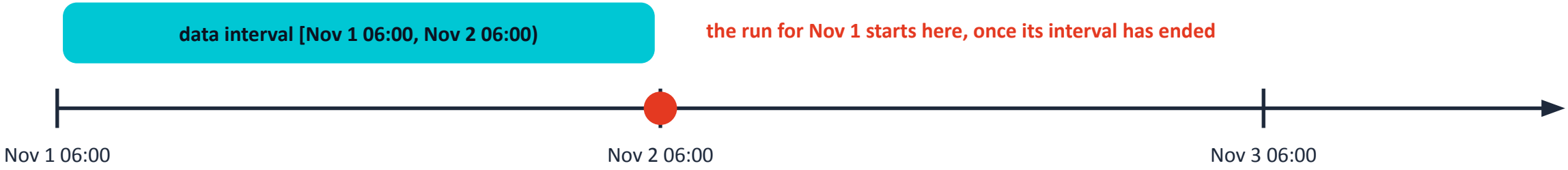
SECTION 3 · 30–45 min

Scheduling, intervals & templating

Demo DAG `03_scheduling_and_templating`



The data interval: the idea that confuses everyone



logical_date = ds = Nov 1 06:00: the start of the interval, not the moment the run happened

```
@dag(
    schedule="0 6 * * *",     # cron | "@daily" | timedelta
                              # | Timetable
    start_date=pendulum.datetime(2026, 11, 1, tz="UTC"),
    catchup=True,             # backfill every missed interval
    max_active_runs=1,
)
def daily_report() -> None: ...
```

- start_date: static and timezone-aware. Never datetime.now().
- The run for [D, D+1) starts at D+1, when day D's data is complete.
- catchup=True + unpause = backfill: one run per interval since start_date. True is the 2.x default.
- Manual trigger: logical_date is the moment you clicked.
- No schedule argument means daily. Pass schedule=None for manual only.

Templating and params: the same context, two syntaxes

variable	example	meaning
{{ ds }}	2026-11-01	logical date as YYYY-MM-DD
{{ logical_date }}	2026-11-01T06:00:00+00:00	start of the data interval (datetime)
{{ data_interval_start }} / _end	… 11-01 06:00 / … 11-02 06:00	bounds for your WHERE clause
{{ run_id }}	scheduled__2026-11-01T06:00	unique per DAG run
{{ params.region }}	eu	user input from the Trigger form
{{ var.value.name }} / {{ conn.x }}	Databurst	Variables and Connections
{{ macros.ds_add(ds, 7) }}	2026-11-08	date helpers, timedelta, json
{{ ti.xcom_pull(task_ids="t") }}	42	XCom of another task

execution_date, prev_ds and next_ds still render in 2.9 but log a deprecation warning, and Airflow 3 removes them. Use logical_date and data_interval_start / _end.

Demo 3: Trigger DAG w/ config, change region, then open the task → Rendered Template tab.

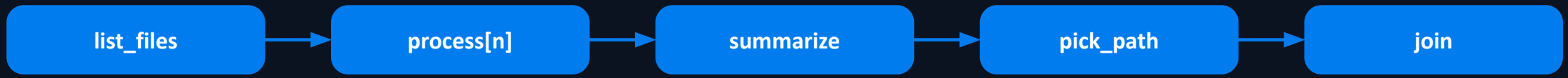
```
# in Python tasks: no Jinja
@task
def t() -> None:
    ctx = get_current_context()
    ctx["ds"]
    ctx["data_interval_start"]
    ctx["params"]["region"]
```

```
# params = typed inputs with a form
params={
  "region": Param("eu",
      enum=["eu", "us", "apac"]),
  "limit": Param(100, type="integer",
      minimum=1),
}
# Jinja renders in template_fields:
# sql, bash_command, @task arguments
```

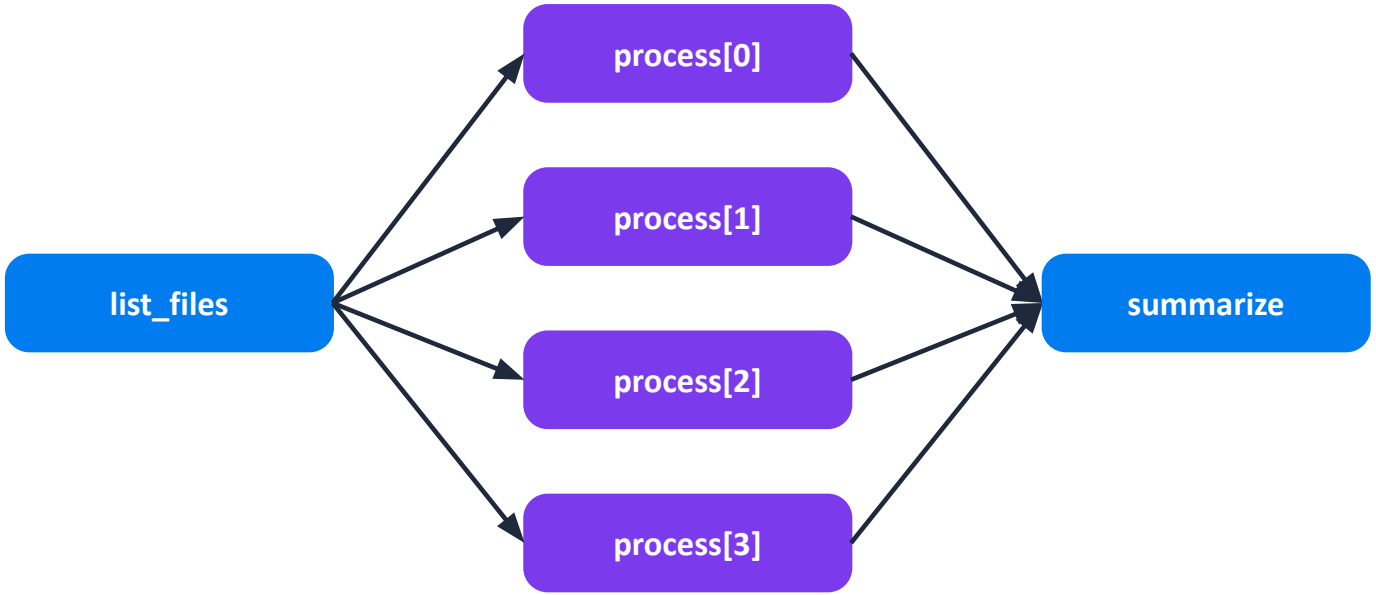
SECTION 4 · 45–60 min

Control flow: mapping, branching, groups

Demo DAG `04_control_flow`



Dynamic task mapping: one task per input, decided at runtime




```
@task
def list_files() -> list[str]: ...

@task
def process(file: str, fmt: str) -> int: ...

sizes = (process
    .partial(fmt="parquet")   # same for all
    .expand(file=list_files()))  # per item

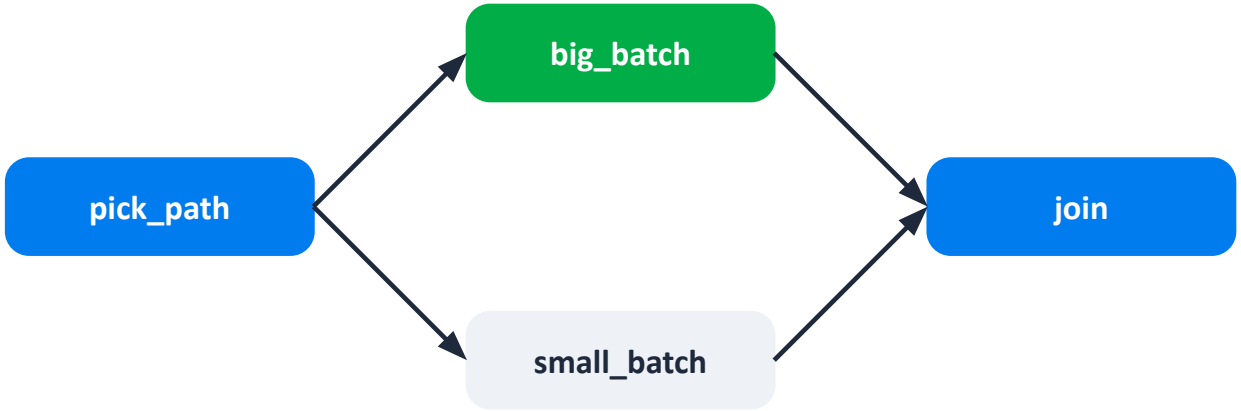
@task
def summarize(sizes: list[int]) -> int:
    return sum(sizes)

summarize(sizes)
```

returns a list → one mapped task per element → the next task receives the list of results

- expand() on a list, dict or the output of another task; expand_kwargs() for several varying arguments; .map() to transform in between
- Replaces “for each file create an operator” at parse time: that pattern cannot react to what exists today, mapping can

Branching and trigger rules



skipped

```
@task.branch
def pick_path(total: int) -> str:
    return "big_batch" if total > 40 else "small_batch"

join = EmptyOperator(task_id="join",
    trigger_rule="none_failed_min_one_success")

@task.short_circuit          # False → skip everything downstream
def only_on_weekdays() -> bool: ...
```

trigger_rule	the task runs when…
all_success (default)	every upstream succeeded; one skip skips me too
none_failed_min_one_success	nothing failed and at least one ran: the join after a branch
all_done	all upstream finished, whatever the state: cleanup
one_success / one_failed	as soon as one upstream succeeds / fails
all_failed / none_skipped	every upstream failed / none was skipped
always	no dependency check at all

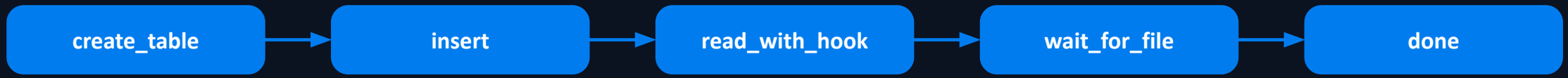
Task groups: @task_group wraps related tasks into one collapsible node in the UI; task_ids become group.task.

Demo 4: trigger it, expand the mapped task in Grid, find the skipped branch in Graph, open the reporting group.

SECTION 5 · 60–75 min

Connections, hooks & sensors

Demo DAG `05_connections_and_sensors`



Talking to the outside world

Connection

Host, port, login, password, extra JSON for one external system, under a conn_id. Stored encrypted (Fernet) in the metadata DB, or in a secrets backend (Vault, AWS SM…), or as AIRFLOW_CONN_<ID> env var.

Variable

A key/value config setting: a bucket name, a threshold, a feature flag. Admin → Variables. Read it inside the task, not at module top level: the file is re-parsed every ~30 s.

Hook

A Python client built from a Connection: PostgresHook, S3Hook, SlackHook. Use hooks in @task functions when no operator fits.

Provider package

apache-airflow-providers-<x>: operators + hooks + sensors + connection types for one system. The Docker image ships ~20; pip install the rest in your own image.

```
insert = SQLExecuteQueryOperator(
    task_id="insert", conn_id="warehouse",
    sql="INSERT INTO greetings (name) VALUES (%(name)s)",
    parameters={"name": "{{ var.value.greeting_name }}"})

@task
def read():
    hook = PostgresHook(postgres_conn_id="warehouse")
    rows = hook.get_records("SELECT * FROM greetings")
```

Sensors: waiting without wasting a worker

`mode="poke"`

The task holds its worker slot and sleeps between checks. Fine for waits under a minute.

`mode="reschedule"`

The task releases the slot and is rescheduled at poke_interval. Good for long waits with few sensors.

`deferrable=True`

The task suspends itself; the triggerer awaits the condition in asyncio and resumes it. Thousands of waits on one small process.

```
wait_for_file = FileSensor(
    task_id="wait_for_file", fs_conn_id="fs_data",
    filepath="ready.flag", poke_interval=5, timeout=600,
    mode="reschedule")

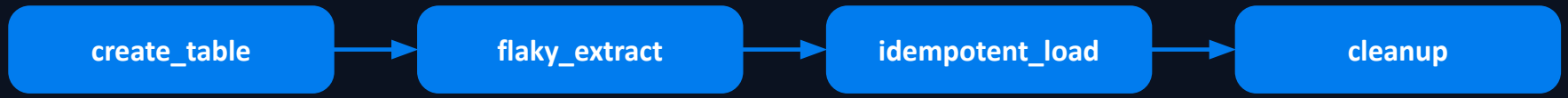
wait_30s = TimeDeltaSensorAsync(    # deferrable: the
    task_id="wait_30s",             # wait runs on the
    delta=timedelta(seconds=30))    # triggerer
```

- Common sensors: FileSensor, S3KeySensor, SqlSensor, ExternalTaskSensor, HttpSensor, @task.sensor
- Always set timeout; a sensor that waits forever hides an upstream failure
- Demo 5: trigger, then on the host run touch data/ready.flag and watch wait_for_file turn green; wait_30s shows the deferred (purple) state

SECTION 6 · 75–90 min

Production patterns & wrap-up

Demo DAG `06_production_patterns`



Habits that keep a DAG boring at 3am

Retries with backoff

retries=3, retry_delay, retry_exponential_backoff=True in default_args. Most failures are transient.

Idempotent tasks

Re-running a day must give the same result: delete+insert by interval, overwrite the partition, upsert on a key.

Lean top level

The file is re-parsed every ~30 s by the scheduler: no DB calls, API calls or Variable.get() outside tasks.

Alert on failure

on_failure_callback on the DAG or default_args → Slack / PagerDuty. Also dagrun_timeout and execution_timeout.

Small XCom, static dates

Pass references, not data. start_date static and tz-aware. max_active_runs=1 when runs must not overlap.

Setup and teardown

@setup / @teardown: create the cluster or temp table, and tear it down even when the work in between failed.

Testing, deploying and scaling

Test locally

```
# does it parse?  (run this in CI)
airflow dags list-import-errors
# run one DAG in-process, no scheduler needed
airflow dags test 06_production_patterns
# or at the bottom of the file:
production_patterns_dag = production_patterns()
if __name__ == "__main__":
    production_patterns_dag.test()
```

Deploy

- DAGs are just files in the dags folder: bake them into your image, sync them with git-sync, or mount a shared volume
- Build your own image on apache/airflow with the providers you need; never pip install at container start

Scale

LocalExecutor

tasks as subprocesses of the scheduler host. Our stack today. Fine for small teams.

CeleryExecutor

a pool of worker machines fed by Redis/RabbitMQ; the official compose default.

KubernetesExecutor

one pod per task; isolation and per-task resources, slower start.

Managed

Astronomer, AWS MWAA, Google Cloud Composer: same DAGs, someone else runs the boxes.

Looking ahead: what changes in Airflow 3

Airflow 2	Airflow 3
from airflow.decorators import dag, task	from airflow.sdk import dag, task, DAG, Param…
from airflow.operators.bash import BashOperator	from airflow.providers.standard.operators.bash import …
execution_date, prev_ds, next_ds…	removed: logical_date, data_interval_start / end
webserver (Flask) + scheduler parsing DAGs	api-server (React UI, REST v2) + mandatory dag-processor
workers read and write the metadata DB	Task SDK: workers only talk to the Execution API
default schedule: daily	default schedule: None (manual)
cron run fires after its data interval	cron run fires at the cron time; intervals are opt-in
SubDAGs, SLA misses, pickling	removed; use task groups, deadline alerts
one version of a DAG	DAG versioning: each run shows the code it ran

WRAP-UP

What you can do now

- Explain the components and read the Grid, Graph and log views
- Write a DAG with @dag / @task and pass small results with XCom
- Choose a schedule, reason about data intervals, backfill and template dates
- Map tasks over runtime inputs, branch, and pick the right trigger rule
- Use Connections, Variables, hooks and sensors without secrets in code
- Ship it: retries, idempotency, callbacks, setup/teardown, dags test

Keep going

- airflow.apache.org/docs: Core Concepts, then Best Practices
- Astronomer guides and the free Astronomer Academy
- Apache Airflow Slack: #user-troubleshooting
- This repo: ./databurst.sh and the six DAGs

Exercise for tomorrow: add a DAG that reads the warehouse table from section 5, maps over its rows, and loads a daily summary idempotently.